Experiment - 10

Applying Performance Optimization Techniques (Indexing, Subforms, Query Tuning) to a Zoho Creator Application

ROSHAN.K
241401083 B.Tech CSBS

AIM

To apply and demonstrate performance optimization techniques, namely indexing, subforms and query tuning, in a Zoho Creator application.

INTRODUCTION

Performance optimization in low-code applications means designing forms, workflows and scripts so that they use data and processing only as far as needed. Although the platform handles much of the underlying work, the way a form is structured and a workflow is written still decides how efficiently the application runs.

In a small application, retrieving extra records or processing data repeatedly may go unnoticed. As the number of bookings, guests and users grows, the same design can slow down searches, reports and form submissions. This experiment studies three techniques that keep an application responsive as it grows: indexing, subforms and query tuning.

Application Used

The practical uses the Hotel Booking application in Zoho Creator, in which guests make room reservations. Deluge scripting is used for the form workflow that sends a booking confirmation email. No benchmark was performed, so the benefits are described in terms of reduced data retrieval and processing, not as measured execution times.

Performance Optimization Techniques

Figure 1 gives an overview of the three techniques and their combined effect. It is a conceptual diagram, not a screenshot.

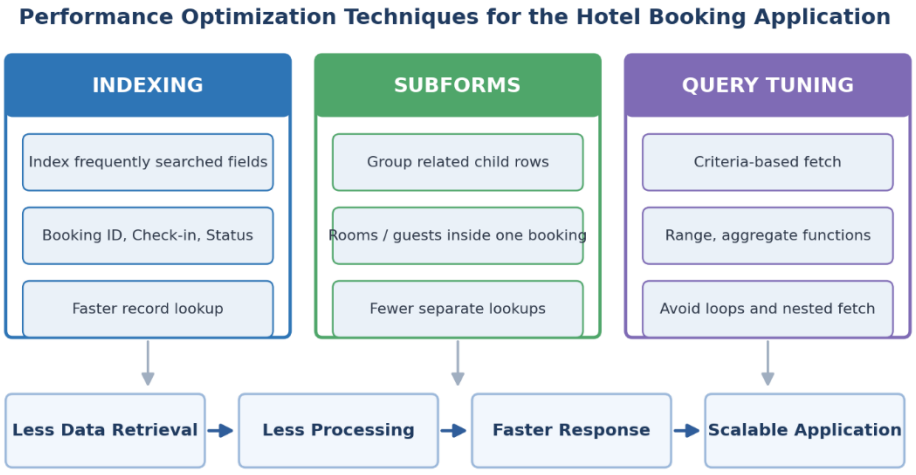


Figure 1: Performance Optimization Techniques (Conceptual Diagram)

241401083

Technique	Idea	Use in the Hotel Booking Application
Indexing	Make frequently searched fields quick to look up	Search by Booking ID, Check-in Date and Status
Subforms	Keep related child rows inside the parent record	Room details inside a single booking
Query tuning	Retrieve only the data that is needed	Fetch the current booking by ID instead of all bookings

Table 1: Optimization Techniques and Their Use

1. Indexing

An index is a lookup structure on a field, similar to the index of a book. Without it, a search for one booking may have to check every record. With it, the system can jump directly to the matching record. Figure 2 illustrates the difference.

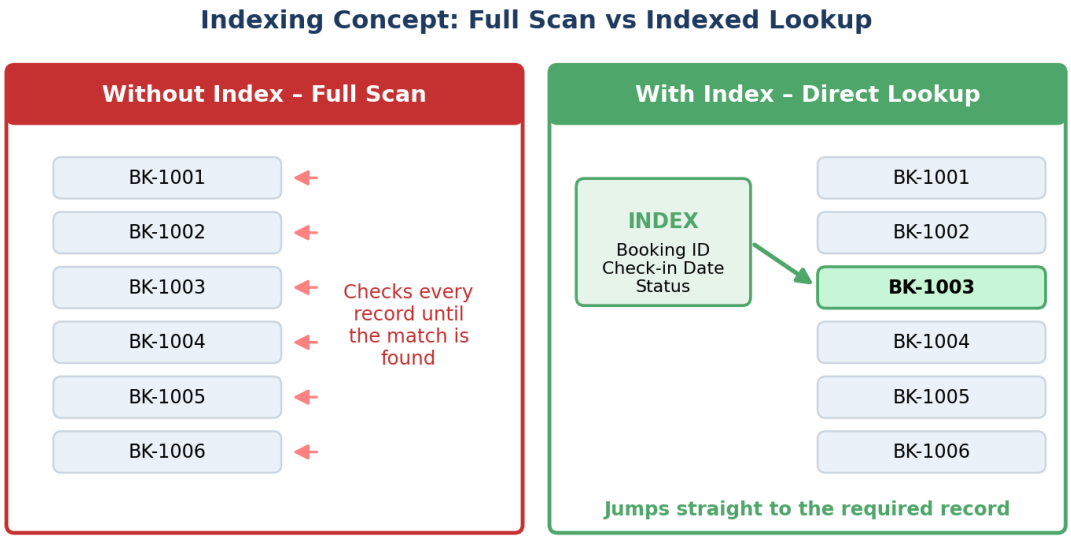


Figure 2: Full Scan vs Indexed Lookup (Conceptual Diagram)

- **Good candidates:** fields that are searched, filtered or used in criteria very often, such as Booking ID, Check-in Date and Status.
- **Poor candidates:** fields that are rarely searched or that change constantly, since maintaining an index has its own cost.
- **Design support:** using the same fields in report filters and Deluge criteria lets an index (where available) give the best benefit.

The availability and configuration of indexing in Zoho Creator depend on the platform and the plan of the account. Indexing is therefore explained here as a technique and is not claimed as enabled in this application.

2. Subforms

A subform is a form embedded inside another form, so that one parent record can hold many related rows. In the Hotel Booking application, a booking (parent) can contain several rooms (rows) in a single Room Details subform, as shown in Figure 3.

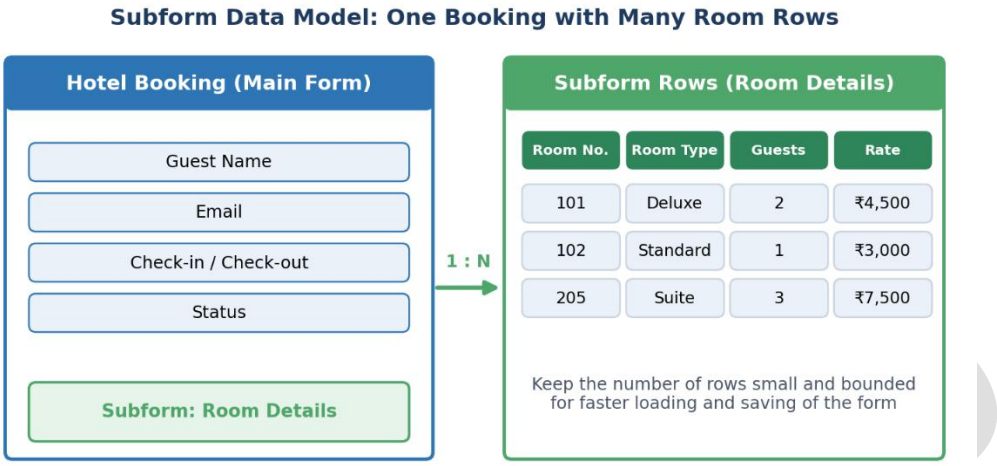


Figure 3: Subform Data Model for Room Details (Conceptual Diagram)

Aspect	Subform	Separate form with lookup
Best for	Small, bounded child rows that belong to one parent	Large or independent sets of records
Data entry	Parent and child rows entered together on one screen	Child records added separately
Performance	Fewer separate lookups; very large row counts slow the form	Scales better for large data; needs a lookup to connect
Hotel example	Rooms in one booking	Guest history across many bookings

Table 2: Subform vs Separate Form

To keep forms fast, the number of subform rows should be limited, only the necessary fields should be included, and large histories should be kept in a separate form.

3. Query Tuning

Query tuning means writing Deluge queries so that the platform retrieves and processes the least data needed. The key technique used in this experiment is criteria-based record fetching: instead of retrieving every record of a form, a criteria is given so that only the required record is returned. Figure 5 compares the two approaches.

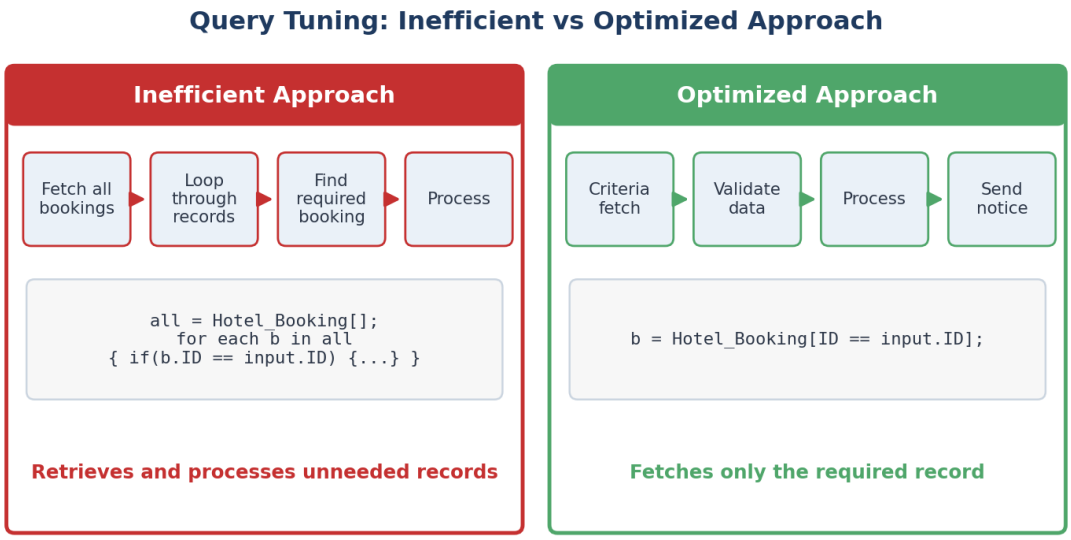


Figure 4: Inefficient vs Optimized Query (Conceptual Diagram)

Illustrative inefficient code (not used in the application):

```
allBookings = Hotel_Booking[];
for each b in allBookings
{
    if(b.ID == input.ID)
    {
        // process the required booking
    }
}
```

Optimized code used in the workflow:

```
booking = Hotel_Booking[ID == input.ID];
```

Further query-tuning practices are summarised below.

Practice	Explanation and Example
Criteria-based fetch	Add a criteria to every fetch, for example Hotel_Booking[Status == "Confirmed"]
Limit the records	Use a range to fetch only the first few records when the full list is not needed, for example [Status == "Confirmed"] range from 1 to 10
Aggregate functions	Use built-in functions such as count() and sum() instead of looping, for example Hotel_Booking[Status == "Confirmed"].sum(Total_Amount)
Avoid nested fetch	Do not fetch records inside a loop; fetch once before the loop or use a better criteria
Avoid unnecessary loops	Do not fetch all records and search through them when a criteria can find the record directly
Keep workflows simple	Avoid repeated or deeply nested operations in a single workflow

Table 3: Query-Tuning Practices

PROCEDURE

1. Open the Hotel Booking application in Zoho Creator in Edit mode.
2. Open the Design section and review the Hotel Booking form and its Room Details subform.
3. Navigate to Workflow, open Form workflows and select the Hotel Booking form.
4. Open the workflow that runs on successful form submission and edit its Deluge script.
5. Replace any fetch of all records with the criteria-based fetch Hotel_Booking[ID == input.ID].
6. Add a check on the email field and send the confirmation email only when the email is available.
7. Save the script and submit a test booking to verify the result.



Figure 5: Form Workflow of the Hotel Booking Application

Deluge Code

The Deluge code of the optimized workflow is as follows:

```
// Fetch the current booking record
booking = Hotel_Room_Booking[ID == input.ID];
// Validate that the email field is not blank
if(booking.ID != null)
{
	if(booking.email_address == null || booking.email_address.trim() == "")
	{
		info "Email address is missing for this booking";
	}
	else
	{
		// Prepare and send the confirmation email
		sendmail
		[
			from :zoho.adminuserid
			to :booking.email_address
			subject :"Your Hotel Booking is Confirmed"
			message :"<p>Dear Guest,</p><p>Your room booking has been
confirmed. We look forward to welcoming you.</p><p>Thank you for choosing our
hotel.</p>"
		]
	}
}
```

The first statement fetches only the booking whose ID matches the submitted record. If its email field is blank, an informational message is displayed; otherwise a confirmation email is sent to the guest.

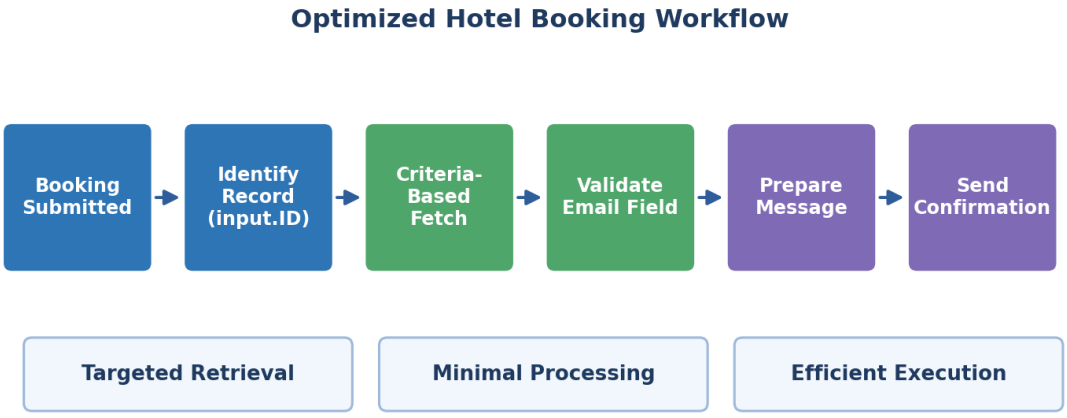


Figure 6: Optimized Hotel Booking Workflow (Conceptual Diagram)

Performance Analysis

Approach	Steps
Before (inefficient)	Fetch all bookings → loop through every record → find the required booking → process
After (optimized)	Fetch the required booking using its ID → process directly
Effect of indexing	The criteria on ID or Status can be resolved by a direct lookup instead of a full scan (where indexing is available)
Effect of subforms	Room rows load and save with their booking, which avoids separate lookups for small row counts

Table 4: Comparison of Approaches

The optimized approach retrieves and processes less data, so the workflow stays efficient as the number of bookings grows.

Best Practices

- Always use a criteria when fetching records; never fetch a whole form without a need.
- Use frequently searched fields in criteria and reports, and index them where available.
- Use subforms only for small, bounded child rows; keep large data in separate forms.
- Prefer built-in aggregate functions to manual loops, and avoid fetching inside loops.
- Keep forms and workflows simple, and review them as the data grows.

CONCLUSION

Performance optimization in Zoho Creator depends on how forms and workflows are designed as well as on the platform. In this experiment, indexing, subforms and query tuning were studied for the Hotel Booking application, and criteria-based fetching, Hotel_Booking[ID == input.ID], was used so that only the required record was retrieved and processed. Together these techniques reduce unnecessary data retrieval and processing and help applications scale as data grows. No benchmark was performed, so the benefits are described in terms of reduced retrieval and processing.